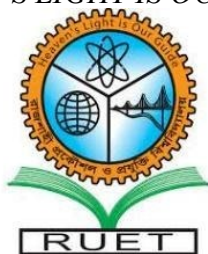


HEAVEN'S LIGHT IS OUR GUIDE



RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF MECHATRONICS ENGINEERING, 22Series

Course No: MTE 3102

Course Title: Control Systems Sessional

Experiment No: 02

Experiment Name: Design of PID controller for DC motor by using trial & error method and analysis their performance

Remarks

--

Submitted By Name: Md. Sadif Hossain Rit ID: 2208002	Submitted To Md. Faisal Rahman Badal Assistant Professor Department of Mechatronics Engineering Rajshahi University of Engineering & Technology, Rajshahi-6204.
---	--

Submission Date: 29 April,2026.

Experiment No: 03

Experiment Name: Design of an LQR controller for a given system using trial & error method.

Objectives:

1. To design PID controller, PD controller, and PI controller for given systems using the Root Locus method.
2. To evaluate and analyze the performance of these controllers based on their Root Locus plots.
3. To adjust and optimize the PID controller parameters to achieve the desired system performance.

Theory:

Linear Quadratic Regulator (LQR):

The Linear Quadratic Regulator (LQR) is an optimal control strategy designed for linear systems to minimize a quadratic cost function, typically representing a trade-off between tracking performance and control energy. It provides a systematic methodology for calculating a feedback gain matrix K that ensures the stability and optimality of the system's state regulation or tracking [2].

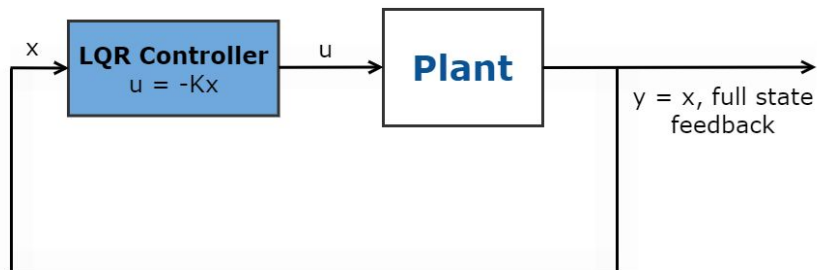


Fig 3.1: Schematic of an LQR Controller [3].

Proportional-Integral-Derivative (PID):

A Proportional-Integral-Derivative (PID) controller is a three-term feedback mechanism that generates a control signal based on the error between a desired setpoint and a measured process variable [1].

1. **Proportional (P):** Provides an action proportional to the current error.
2. **Integral (I):** Reduces steady-state error by accumulating past errors.
3. **Derivative (D):** Improves transient response by predicting future error based on its current rate of change [1].

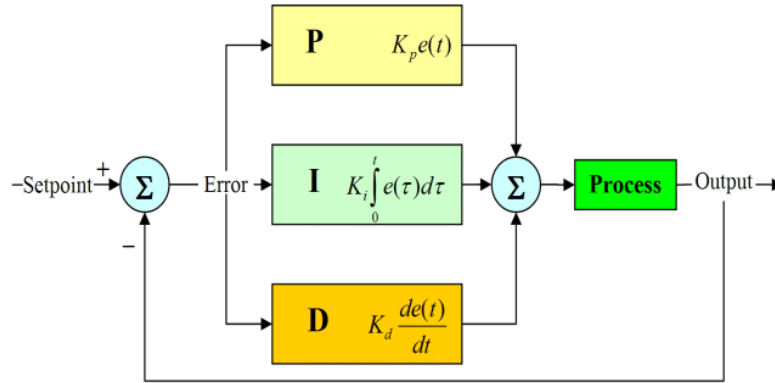


Fig 3.2: Block diagram for PID controller [4].

Root Locus:

The Root Locus method is a graphical technique used in control system design to show how the roots (poles) of the closed-loop characteristic equation move in the complex s-plane as a specific parameter, typically the loop gain K , varies from zero to infinity. It allows designers to evaluate system stability and transient performance visually by observing the trajectories of these poles.

Trial and Error Method:

In the context of LQR design, **trial and error** refers to the iterative process of manually adjusting the weighting matrices Q (state penalty) and R (control penalty) to meet specific performance criteria such as rise time, overshoot, or settling time [2].

Because the relationship between the weights in Q and R and the resulting time-domain performance is often non-linear and complex, designers frequently start with Bryson's Rule or an identity matrix and then "tune" the diagonal elements until the system response is optimized [2].

Required Apparatus:

1. MATLAB 2024a.
2. Laptop.

System model:

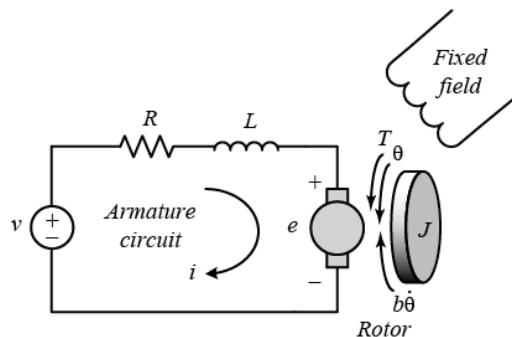


Fig 3.3: DC motor circuit [5].

Transfer Function

$$\text{Transfer Function of DC Motor, } \frac{\theta'(s)}{V(s)} = \frac{k}{(R+sL)(js+c)+k^2}$$

$$\text{Transfer Function of PID Controller} = \frac{k_D \left(s^2 + \frac{kp}{k_D} s + \frac{k_I}{k_D} \right)}{s}$$

$$\text{Transfer Function of PI Controller} = \frac{(skp+ki)}{s}$$

$$\text{Transfer Function of PD Controller} = k_D \left(s + \frac{kp}{k_D} \right)$$

Code:

```
clc;
clear;
close all;

% System Parameters (DC Motor)

J = 0.01;
b = 0.1;
K = 0.01;
R = 1;
L = 0.5;

s = tf('s');

% Transfer Function
TF = K / ((J*s + b)*(L*s + R) + K^2);

%% ----- PID Controller -----

Kp_pid = 800;
Ki_pid = 500;
Kd_pid = 50;

PID = pid(Kp_pid, Ki_pid, Kd_pid);

CL_PID = feedback(PID*TF, 1);
```

```

%% ----- PI Controller -----

Kp_pi = 400;

Ki_pi = 250;

PI = pid(Kp_pi, Ki_pi, 0);

CL_PI = feedback(PI*TF, 1);

%% ----- PD Controller -----

Kp_pd = 1000;

Kd_pd = 20;

PD = pid(Kp_pd, 0, Kd_pd);

CL_PD = feedback(PD*TF, 1);

%% ----- Function to Calculate Delay Time -----

% Defining a small helper function to find the 50% mark

getDelayTime = @(sys) info_with_delay(sys);

%% ----- Step Responses -----

figure;

step(CL_PID, CL_PI, CL_PD);

legend('PID (to 1.0)', 'PI (to 1.0)', 'PD (settles < 1.0)');

title('Closed Loop Step Response Comparison');

grid on;

set(gca, 'FontSize', 12);

lines = findall(gcf, 'Type', 'line');

set(lines, 'LineWidth', 2);

%% ----- Performance Info & Delay Time Calculation -----

controllers = {CL_PID, CL_PI, CL_PD};

names = {'PID', 'PI', 'PD'};

for i = 1:3

    fprintf('\n--- %s ---\n', names{i});

    info = stepinfo(controllers{i});

```

```

% Calculate Delay Time (Time to reach 50% of steady-state value)

[y, t] = step(controllers{i});

final_val = y(end);

delay_index = find(y >= 0.5 * final_val, 1);

delay_time = t(delay_index);

% Display Info

disp(info);

fprintf('DelayTime: %.4f seconds\n', delay_time);

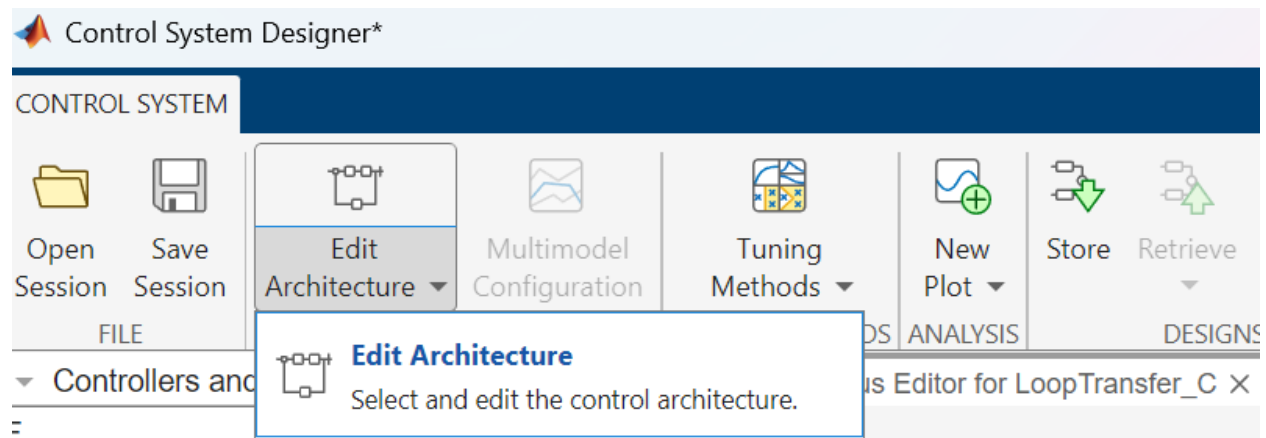
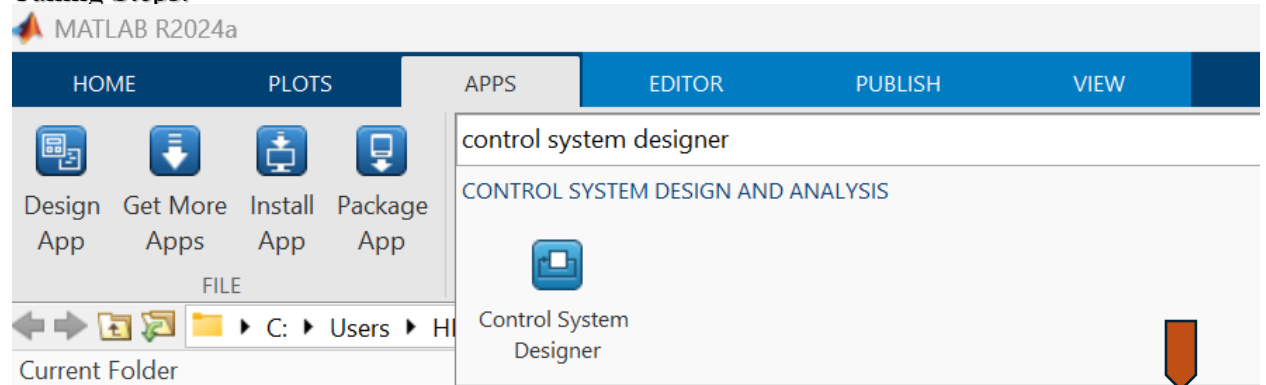
end

%% ----- Root Locus -----

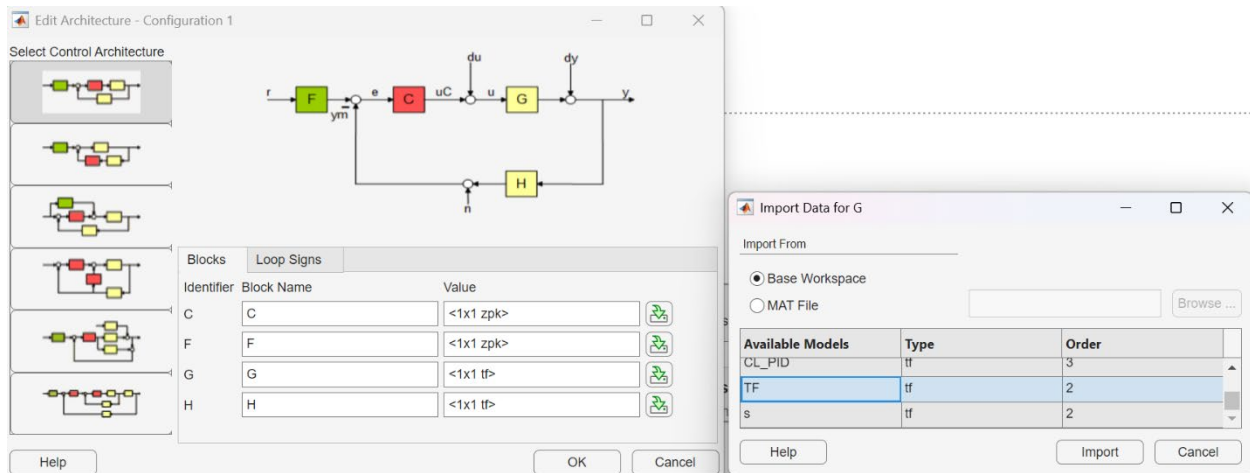
% (Keep your existing root locus code here if needed)

```

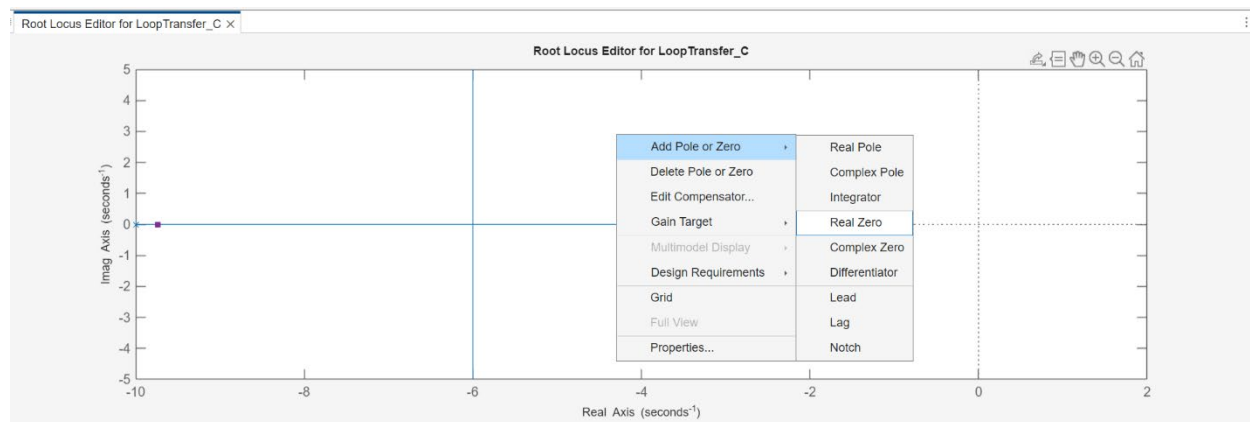
Tuning Steps:



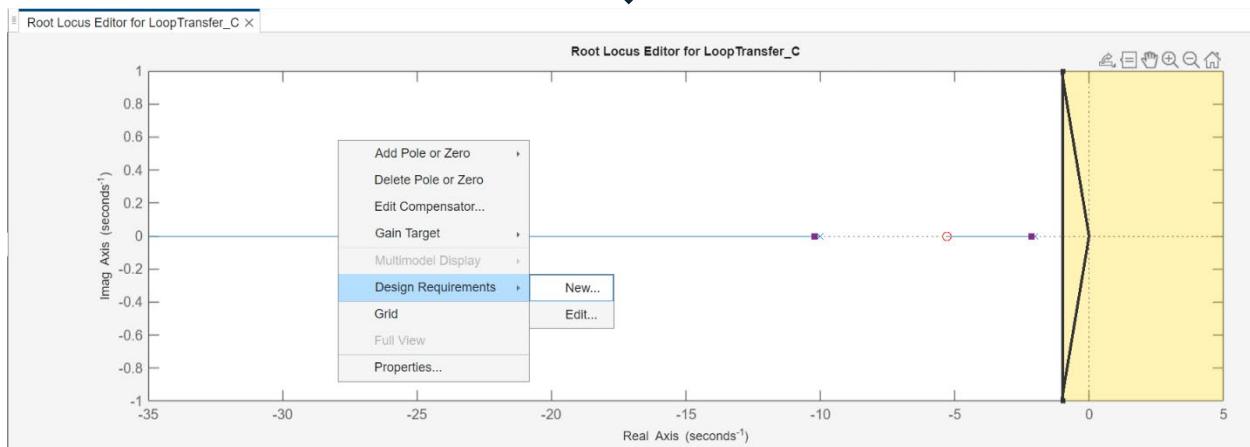
(Selecting Edit Architecture)



(import data for G and chose TF to import and pressing ok)

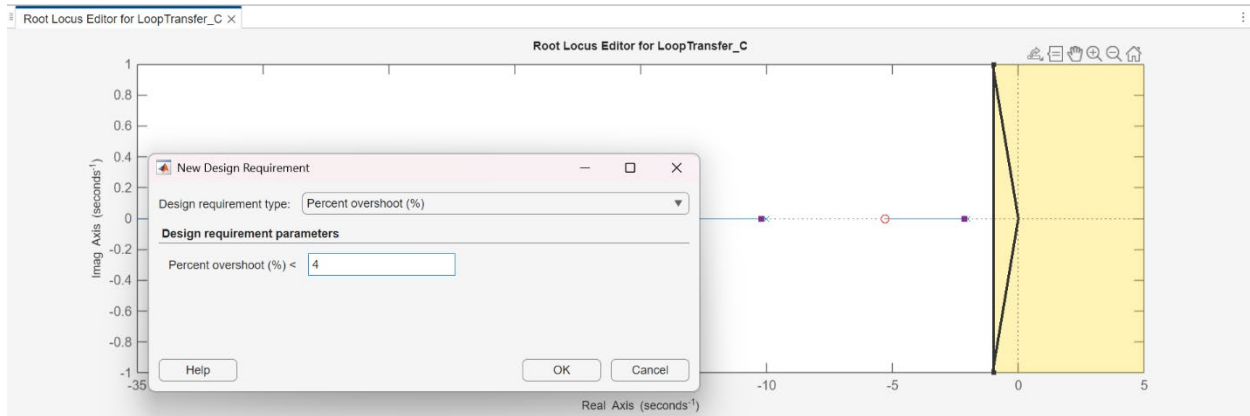
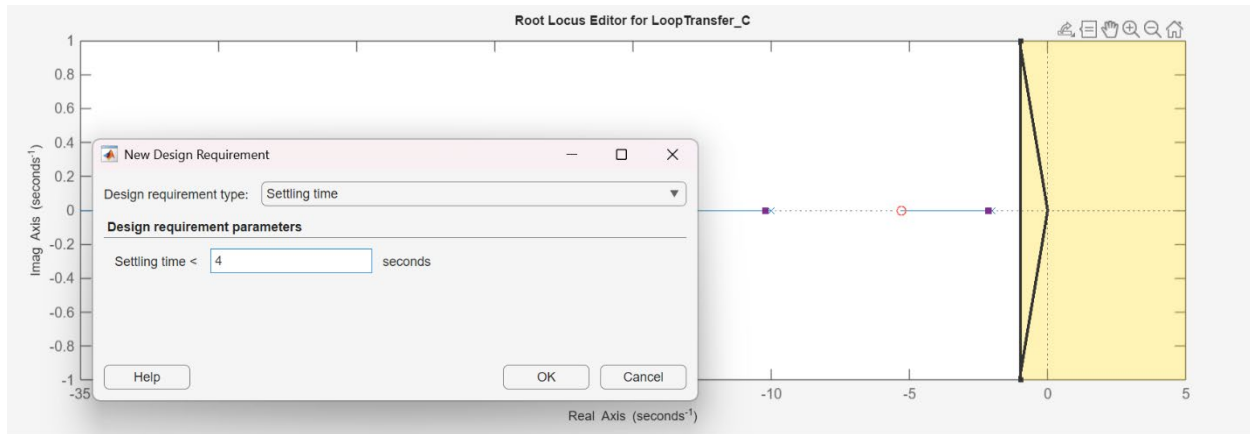


(Adding pole to Real zero)



(Design Requirements to New)





▼ Controllers and Fixed Blocks

F

C



▼ Preview

Tunable Block

Name: C

Sample Time: 0

Value:

0.18882 (s+5.296)

Results:

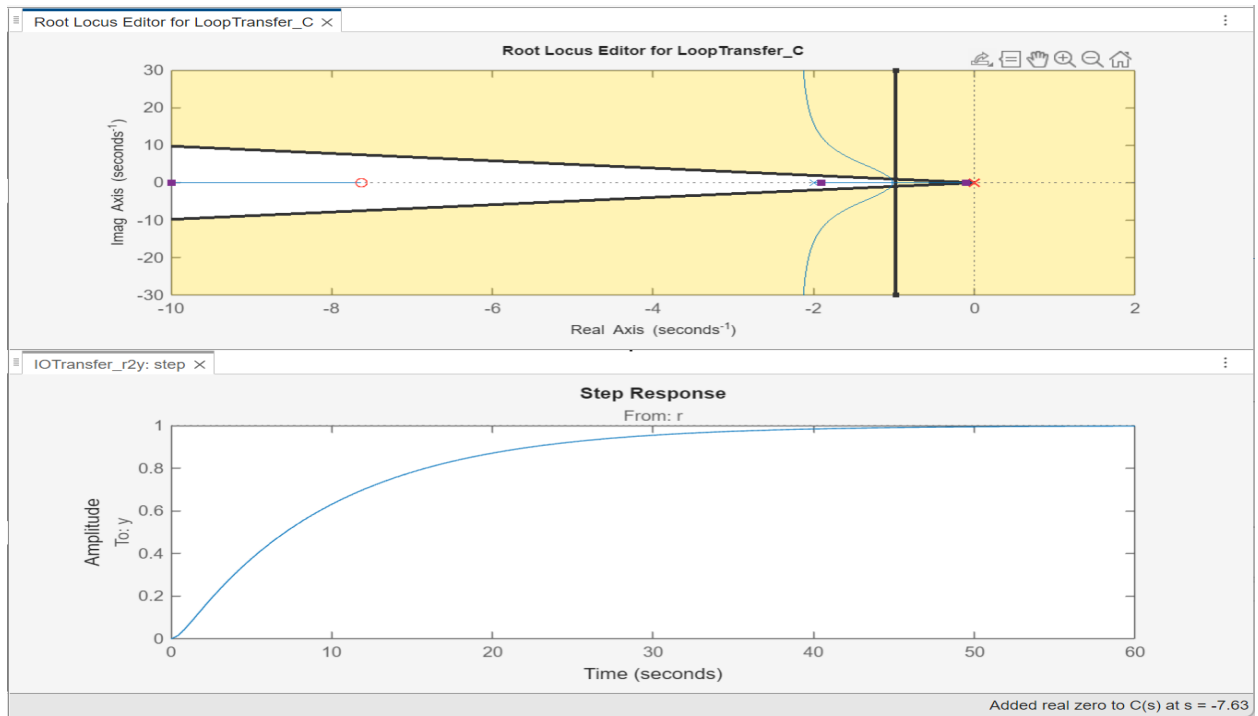


Fig 3.4: Root locus and Step response of PID controller from the editor.

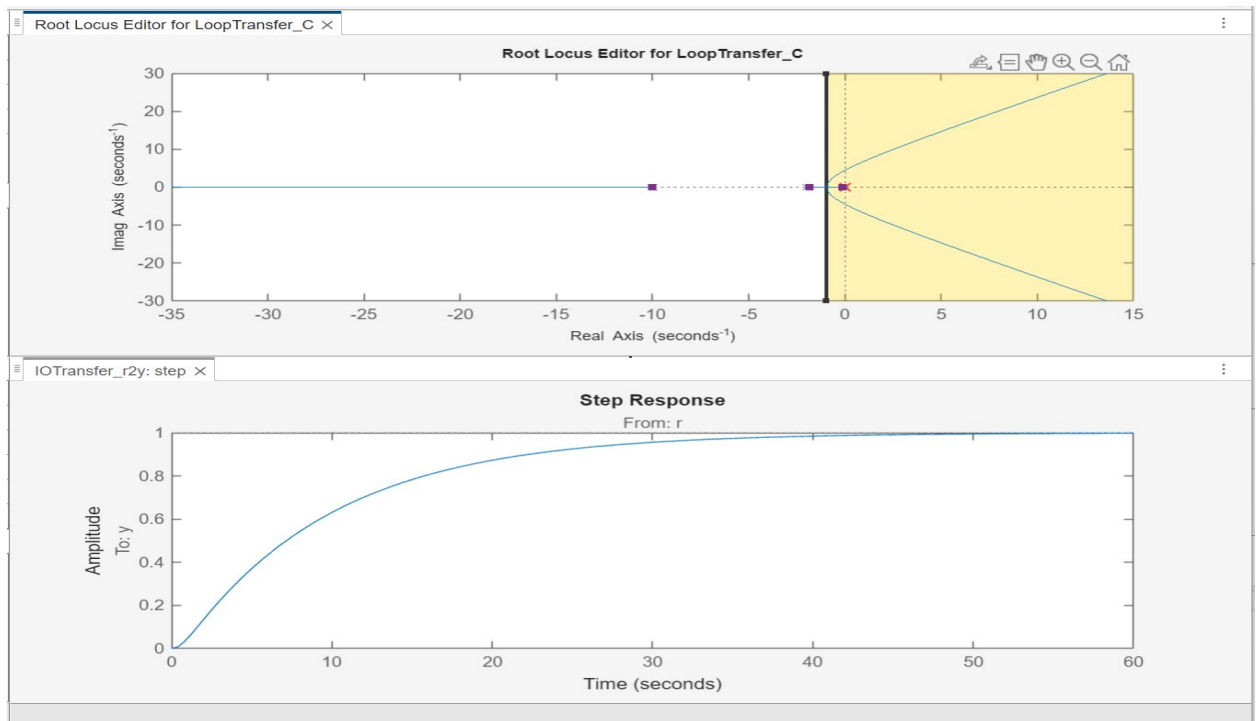


Fig 3.5: Root locus and Step response of PI controller from the editor.

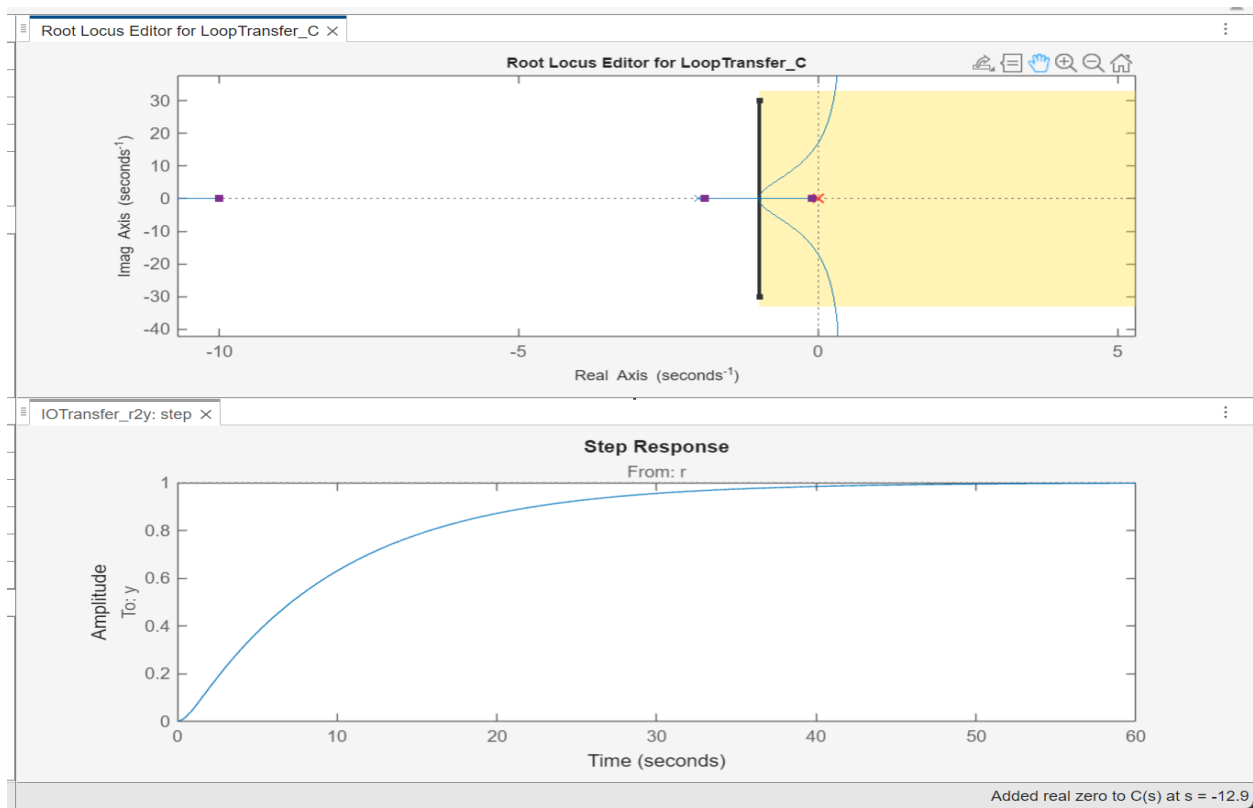


Fig 3.6: Root locus and Step response of PD controller from the editor.

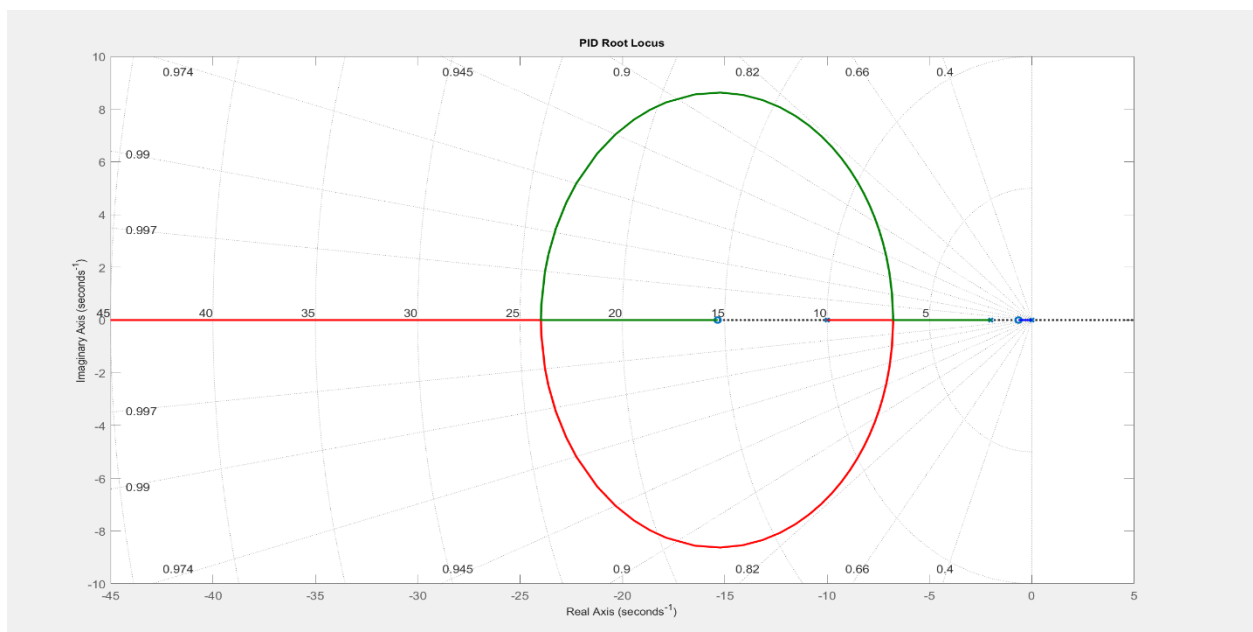


Fig 3.7: PID root locus graph from the code.

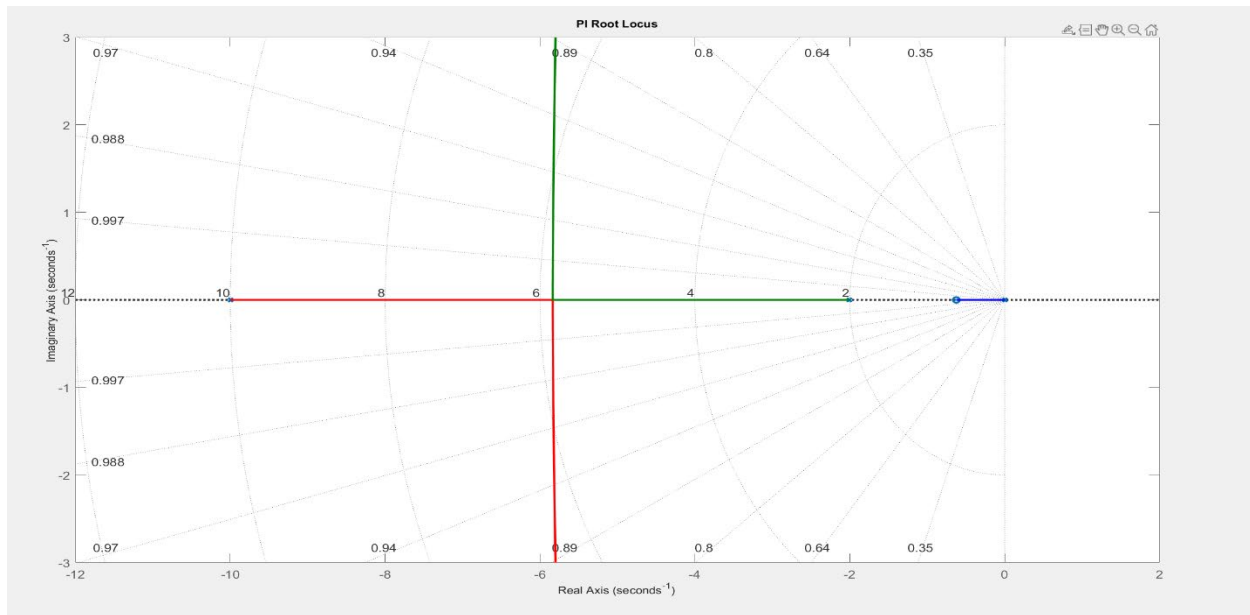


Fig 3.8: PI root locus graph from the code.

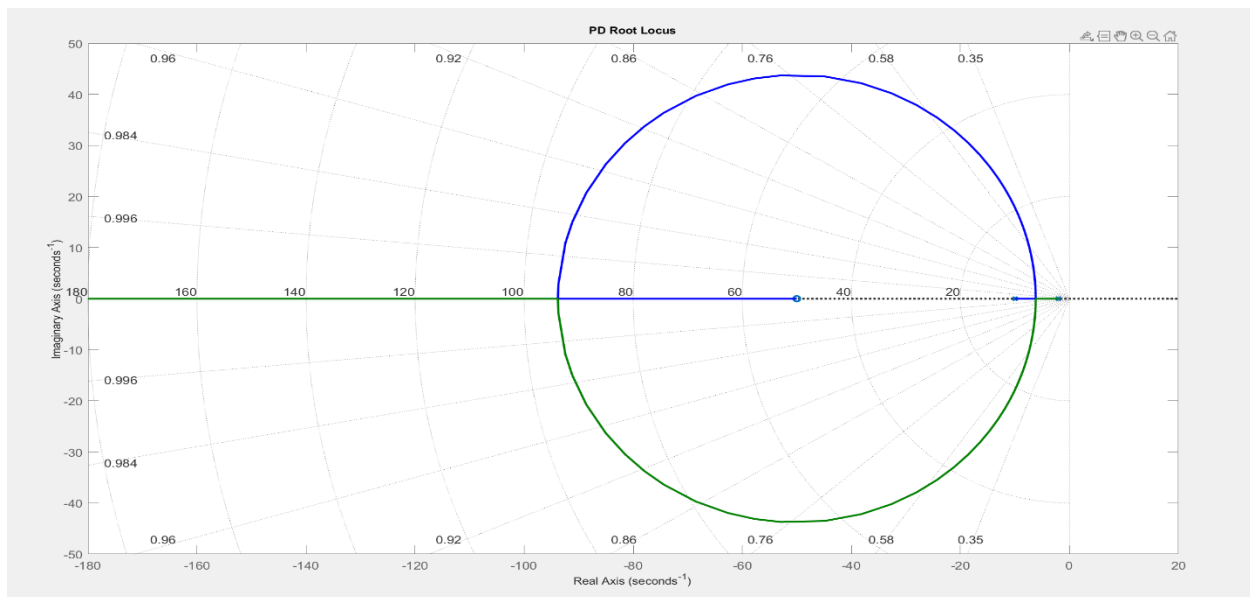


Fig 3.9: PD root locus graph from the code.

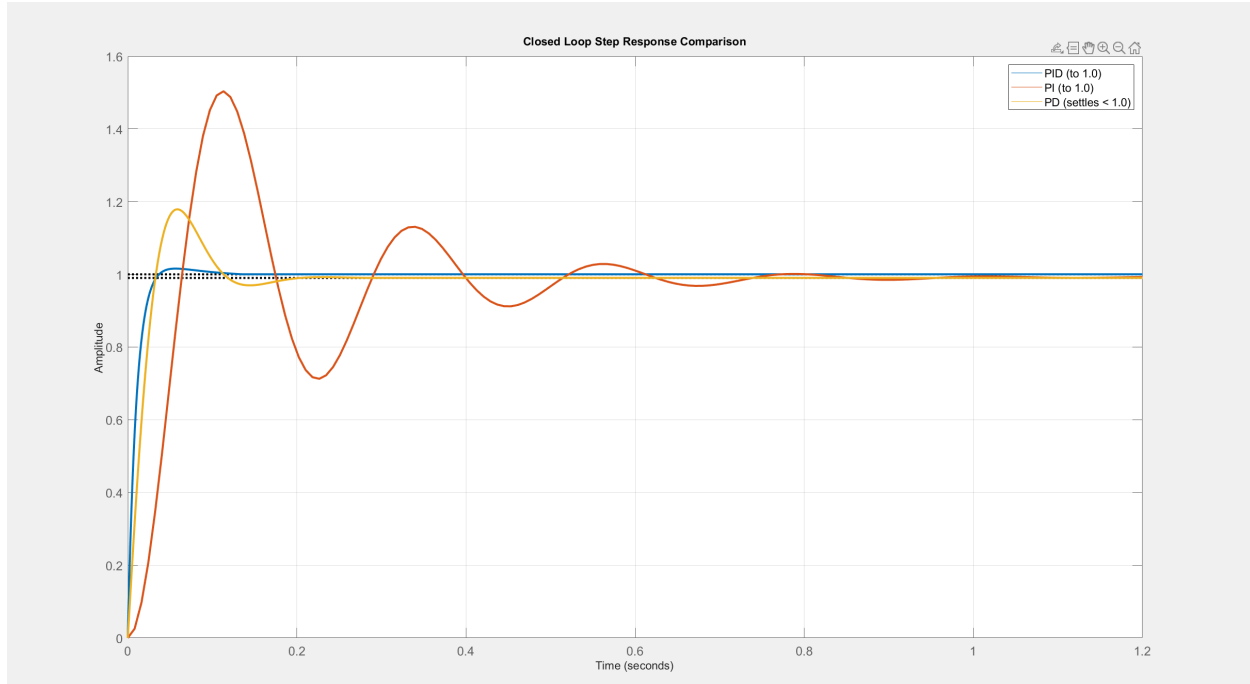


Fig 3.10: Step responses from the code.

Systems	Delay Time(s)	Rise Time(s)	Peak Time(s)	% Overshoot	Settling Time(s)
PI	0.0404	0.0429	0.1133	50.3545	0.7165
PD	0.0142	0.0250	0.0585	0	0.1497
PID	0.0073	0.0200	0.0567	1.5808	0.0311

Discussion and Conclusion:

In this experiment, the control of a DC motor was analyzed using PID, PI, and PD strategies. Initial faults were identified as system instability, characterized by a "runaway" response, and significant steady-state error where the output failed to reach the unit step of 1.0. These faults were attributed to the accidental placement of zeros in the Right Half Plane (RHP) and the lack of integral action in a Type 0 system.

To mitigate these errors, the controller was redesigned to ensure all closed-loop poles remained in the Left Half Plane (LHP). An Integrator was incorporated to eliminate steady-state error, and gains (K_p , K_i , K_d) were increased to overcome the small motor constant ($K = 0.01$). It was concluded that while PD control improved speed, only the PID and PI controllers achieved the target amplitude of 1.0. The PID controller was ultimately determined to be the superior solution, providing the best balance of stability, zero tracking error, and optimized transient performance.

Reference:

[1]. Ang, K. H., Chong, G., & Li, Y. (2005). PID control system analysis, design, and technology. *IEEE Transactions on Control Systems Technology*, 13(4), 559–576. <https://doi.org/10.1109/tcst.2005.847331>

Cited by: 4563.

[2]. Salem, N., Mateen, K., Alharbi, W., & Kamal, J. (2023). Performance of LQR and PID Controllers for RS-550VC Motor Speed Enhancement. 2023 6th International Conference on Intelligent Robotics and Control Engineering (IRCE), 1–6. <https://doi.org/10.1109/irce59430.2023.10255039>

Cited by: 8.

[3]. MathWorks, “What Is Optimal Control?” MathWorks. [Online]. Available: <https://www.mathworks.com/discovery/optimal-control.html>

. [Accessed: Apr. 21, 2026].

[4]. A. Marzouki, “PID Block Diagram: PID stands for Proportional-Integral-Derivative control,” ResearchGate, 2012. [Online]. Available: https://www.researchgate.net/figure/PID-Block-Diagram-PID-stands-for-Proportional-Integral-Derivative-control-A-PID_fig1_316709017

. [Accessed: Apr. 21, 2026].

[5]. University of Michigan, “DC Motor Speed: System Modeling,” Control Tutorials for MATLAB and Simulink (CTMS). [Online]. Available: <http://ctms.engin.umich.edu/CTMS/index.php?example=MotorSpeed§ion=SystemModeling>

. [Accessed: Apr. 21, 2026].